

EJERCICIOS AVIÓNICA SOFTWARE

Antes de desarrollar el contexto de la misión, ya que superar el récord mundial es algo atípico: A Mach 5.2 (enunciado estructuras) y más de 143km de altura los componentes típicos de un cohete de competición fallan: GNSS se bloquea (ya sea por restricciones COCOM de altitud o velocidad o incluso por el efecto Doppler, la temperatura o las vibraciones del lanzamiento), el barómetro deja de medir aproximadamente a 31km de altura cuando se pierde ya más del 99% de la masa de la atmósfera terrestre y son indistinguibles ya los cambios y además la telemetría sufriría problemas porque con frecuencias comerciales puedes tener problemas con la altura entre muchos otros problemas que ahora vamos a describir.

Investigamos el caso más parecido que existe (cohete que sustenta el record de altura, Aftershock II) ya que nos servirá para responder a todas las cuestiones siguientes:

[hoja de cálculo del repositorio: FARADAY 1.xlsx]

(Otro buen caso para hacer un poco de ingeniería inversa es el caso del Meraki III, cohete estudiantil con record de velocidad, 111km)

EJERCICIO 1

apartado a

Voy a definir un marco de decisión para que mi decisión tenga sentido basándome en el proceso de análisis de decisiones del [NASA Systems Engineering Handbook](#) (sección 6.8):

1. Definir criterios que la misión exige.
2. Identificar alternativas.
3. Analizarlas (matriz de decisión).
4. Elegir

(El orden importa, umbrales y pesos se fijan antes de poner a todos los participantes)

1. Definir criterios que la misión exige:

1.1. El ordenador de vuelo tiene que ser capaz de hacer tres cosas durante la misión: Estimar estados, detectar eventos y vigilar su propia salud para ello se hace una preselección de sensores y que exigencias tienen estos respecto al microprocesador (sobre todo que buses necesito y cuantos):

(Se hace ya el ejercicio 2a)

Estos se seleccionan con una herramienta más sencilla que una matriz, simplemente teniendo en cuenta dos cosas: la física de vuelo y la arquitectura del microcontrolador.

Aproximadamente 25 componentes (se les asigna una importancia):

[hoja de cálculo del repositorio: FARADAY 1.xlsx]

IMU (x2, Crítico) se duplica por redundancia, esto se exige incluso en EuRoC y America cup

Acelerómetro de altas g (x1, Crítico): (IMUS se saturan a 16-32 g, principio)

Giróscopo de alto rango (x1, Importante): Evita la saturación si el cohete experimenta un alabeo

Así mismo sabemos que un ordenador de vuelo tiene que hacer tres cosas: estimar estados, detectar eventos y vigilar su propia salud.

Para la matriz de decisión:

- . Umbral (mínimo para completar la función, si no llega no se puntúa)
- . Peso (cuánto importa ese criterio frente a los demás)
- . Nota (lo bien que cada candidato cumple ese criterio, de 1 a 5).

Sobre los modelos STM:

QUE SE HA ESCRITO EN EL CÓDIGO:

1. ESTRUCTURA POR CAPAS

main.c "quiero los datos de todos los sensores"

|

sensors.c inicializa y lee todos; si uno falla, sigue con los demás

|

icm42686.c h3lis200dl.c rm3100.c ms5607.c ina226.c (un driver/sensor)

|

bus.c sabe CÓMO se habla por SPI e I2C (CS, bit de lectura, timeouts)

|

HAL de ST mueve los bits por el periférico del micro

(el main.c se regenera y así distribuido es más práctico por si se cambia algo)

2. Sensor_types.h

```
#ifndef INC_SENSORS_SENSOR_TYPES_H_
#define INC_SENSORS_SENSOR_TYPES_H_
...
#endif /* INC_SENSORS_SENSOR_TYPES_H_ */
```

Estas líneas evitan que el contenido del archivo se procese más de una vez si es incluido accidentalmente múltiples veces en un mismo archivo fuente (.c)

“enum” drivers comunican un error de código y se quedan a la espera:

SENS_OK = 0: La operación se completó con éxito.

SENS_ERR_BUS: Hubo un fallo a nivel de comunicación física

SENS_ERR_ID: El sensor conectado no responde, o el ID leído no coincide con el modelo esperado

SENS_ERR_TIMEOUT: El sensor tardó más de lo permitido en preparar o entregar los datos solicitados.

SENS_ERR_CRC: Los datos internos están corruptos

3. bus.h y bus.c (como se habla con los sensores)

Son el puente entre mi MCU (usando HAL) y los sensores físicos a través de I2C, SPI, etc..

extern SPI_HandleTypeDef hspi1;

extern SPI_HandleTypeDef hspi2;

extern I2C_HandleTypeDef hi2c1;

(esto es para luego usar la variable dentro de otra capa, el sensor concreto)

```
typedef struct {
```

```
    SPI_HandleTypeDef *hspi;
```

```
    GPIO_TypeDef *cs_port;
```

```
    uint16_t cs_pin;
```

```
} spi_dev_t;
```

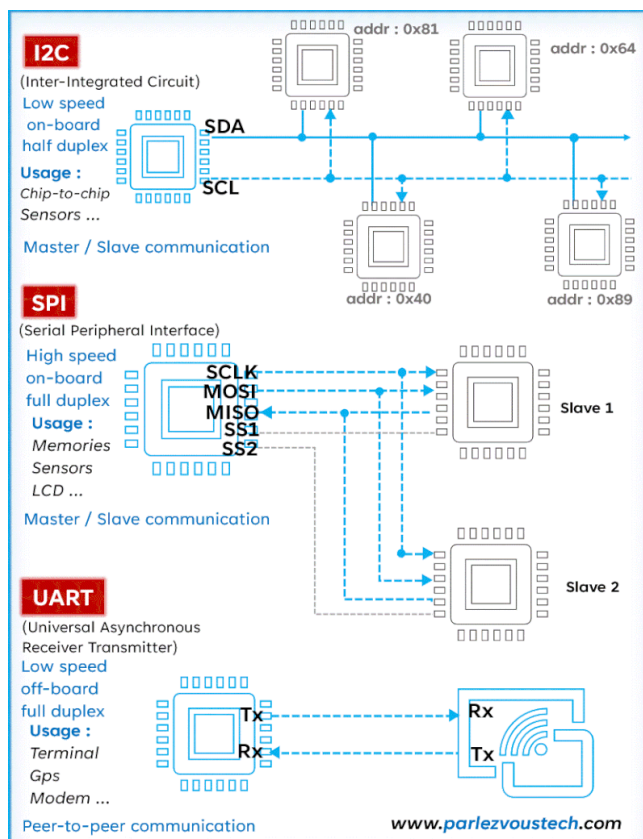
(Un dispositivo SPI = qué bus usa + qué pin es su chip-select, si varios sensores comparten el mismo bus SPI (como el H3LIS y el RM3100), lo único que permite diferenciarlos y hablar con uno u otro es activar su respectivo pin Chip-Select)

En el protocolo SPI, antes de pedir o mandar datos, hay que indicarle al sensor qué registro queremos tocar utilizando el bit 7 de la dirección:

LECTURA (reg | 0x80) poner en HIGH (operación OR sobre bit 7) (al final se sube siempre el CS al terminar para no bloquear el bus)

ESCRITURA (reg & 0x7F) poner en LOW (operación AND sobre bit 7)

Según el sensor se habla diferente por I2C



4. IMU (ICM-42688-P)

```
static int16_t be16(const uint8_t *p) { return (int16_t)((uint16_t)(p[0] << 8) | p[1]); }
```

(static, porque limitamos el alcance solamente a este archivo .c; int16_t con signo ya que experimenta las fuerzas en los dos sentidos)

LA ESCALA:

El conversor analógico-digital interno de la IMU es de 16 bits con signo, cubriendo un rango de -32768 a $+32767$ cuentas. Si configuras la IMU en un rango de $\pm 32g$, la relación matemática es directa:

$$\frac{32768 \text{ cuentas}}{32g} = 1024 \text{ LSB/g}$$

Lo mismo aplica para el giróscopo ($\pm 4000^\circ/\text{s}$), resultando en unas 8.2 cuentas por grado por segundo.

(Cuando un sensor digital, de lo que sea, en este caso un IMU, mide algo en el mundo real, toma una señal analógica continua y la divide en “pasos”, 1LSB equivale exactamente al valor de un paso.

5. H3lis200dl acelerómetro de alta gama

La IMU principal esta configurada en un rango (32g) para tener mucha precisión en vuelo ordinario. Pero en despegue para no saturar y estropear los cálculos de navegación inercial.

Estructura y Registros (hig_dev y #define):

- Configura el bus SPI y el pin de selección de chip (CS).
- Define los registros de control para habilitar los tres ejes, configurar el muestreo a **1 kHz**, activar el filtro **BDU** (evita corrupción de datos al leer) y fijar el rango en $\pm 200g$.
- Especifica los registros de salida (0x29, 0x2B, 0x2D), destacando que **no son contiguos**.

Inicialización (h3lis200dl_init):

1. Verifica el sensor leyendo el registro WHO_AM_I (espera el valor 0x32).
2. Configura CTRL_REG4 para establecer el rango y el BDU.
3. Configura CTRL_REG1 para encender el sensor en modo normal a 1 kHz con sus tres ejes activos.

Lectura de datos (h3lis200dl_read):

1. Realiza tres lecturas individuales por SPI debido a que los registros de los ejes no son consecutivos.
2. Convierte cada byte bruto a un entero con signo (int8_t).
3. Multiplica el valor por el factor de sensibilidad (1.56f) para entregar la aceleración final en unidades de gravedad (g).

6. rm3100.c: El magnetómetro de 24 bits

-Se utiliza el modo POLL (bajo demanda, dispositivo maestro solo solicita datos cuando los necesita, no los pide de manera continua)

-Problema, el microcontrolador maneja tipos de 8,16,32 bits (significa que el tamaño físico de sus circuitos internos, como los registros o los buses de datos, están diseñados para procesar bloques de esa cantidad exacta de dígitos binarios de manera simultánea en un solo ciclo de reloj.

PERO RM3100 entrega el campo magnético en 3 bytes:

guardamos la info en un int32_t, pero el sensor me entrega 24 bits:

Si numero positivo bits de arriba quedan libres pero se sigue leyendo

Si numero negativo (PROBLEMA)

*En informática, para saber si un número es negativo, se mira su **bit más alto** (el último de la izquierda). Si es un 1, es negativo.

Como tu sensor te da 24 bits, ese "bit negativo" cae justo en la posición 24.

Al meterlo en la caja de 32 bits, los espacios que sobran por arriba se rellenan con ceros por defecto.

El procesador ve esos ceros arriba y piensa equivocadamente: *"Oye, como los bits de más arriba son ceros, esto es un número positivo gigante"* (un valor enorme de más de 8 millones).

La solución: Forzar la "Extensión de signo"

Para que el procesador no se equivoque, hay que avisarle de que el número es negativo rellorando los huecos vacíos con **unos (1)** en lugar de ceros. Eso es lo que hace esa línea de código en dos pasos:

1. **if (v & 0x800000):** Revisa si el bit 24 es un 1 (es decir, si el sensor dice que el número es negativo).
2. **v |= 0xFF000000:** Si descubre que sí es negativo, **llena de unos** todos los bits de arriba de la caja de 32 bits.

En el momento en que el procesador ve esos unos arriba, entiende la regla matemática (*complemento a dos*) y dice: *"Ah, vale, es un número negativo real"*, dándote el valor correcto (por ejemplo, -1500 en lugar de un error absurdo de 8000000).

CÓDIGO GENERAL:

Este archivo (rm3100.c) implementa el driver SPI para el magnetómetro **RM3100**, encargado de medir el campo magnético en tres ejes (*X, Y, Z*) para la navegación del cohete.

Sus componentes clave se resumen a continuación:

- **Estructura y Registros (mag_dev y #define):**
 - Configura la conexión por **SPI2** y el pin de selección de chip (CS).
 - Define constantes operativas como los **200 ciclos de muestreo**, la ganancia matemática (75 LSB/ μ T) y un límite de seguridad (*timeout*) de 20 ms.
- **Traducción de 24 bits (be24):**
 - Empaqueta los 3 bytes que entrega el sensor para cada eje en una variable de 32 bits (int32_t).
 - Aplica **extensión de signo** para asegurar que los valores negativos se interpreten correctamente por el microprocesador.
- **Inicialización (rm3100_init):**
 1. Verifica la identidad del sensor leyendo el registro de revisión (espera el valor **0x22**).
 2. Configura los registros de los tres ejes con los 200 ciclos definidos para fijar la sensibilidad de forma estricta.
- **Lectura de datos (rm3100_read):**
 1. **Disparo:** Envía la orden de medir simultáneamente los tres ejes.
 2. **Espera segura:** Revisa cíclicamente el registro de estado hasta que los datos están listos (DRDY), usando un *timeout* para evitar bloqueos si el sensor falla.

3. **Conversión:** Lee los 9 bytes en ráfaga, los procesa con la función `be24()` y los divide entre 75.0 para entregar la orientación final en microteslas (μT).

7. Ms5607.c : El barómetro

Este sensor no nos da la presión en pascals ni la temperatura en grados, sino en número bruto (D_1 y D_2) además hay seis constantes de calibración (C_1 - C_6) Necesita de fórmulas matemáticas complejas con coeficientes de fábrica para arrojar datos reales de presión y temperatura.

1. Funcionamiento sin Mapa de Registros y Constantes

- **Dirección I2C (MS5607_ADDR = 0x77):** Identifica al dispositivo en el bus I2C.
- **Comandos (#define):** En lugar de leer o escribir registros fijos, se envían comandos como `CMD_RESET`, órdenes de conversión de alta resolución (`CMD_CONV_D1_4096` para presión y `CMD_CONV_D2_4096` para temperatura) y la orden de lectura de la memoria interna (`CMD_PROM_READ`).
- **La PROM y los coeficientes (C_1 a C_6):** Cada sensor es único de fábrica. El fabricante quema 6 constantes de calibración en una memoria interna (PROM) que se almacenan en el array `C[]`:
 - C_1 : Sensibilidad de presión.
 - C_2 : Offset de presión.
 - C_3 y C_4 : Controlan la dependencia térmica de la sensibilidad y el offset.
 - C_5 y C_6 : Calibran la lectura de temperatura pura.

2. Inicialización y Validación (ms5607_init)

1. **Comprobación de presencia:** Verifica que el dispositivo responde en la dirección I2C.
2. **Reinicio (Reset):** Envía el comando de reinicio para que el sensor recargue su PROM interna.
3. **Lectura de la PROM:** Lee las 8 palabras de 16 bits de la memoria del sensor (los coeficientes de calibración y un campo final para el CRC).
4. **Verificación CRC (crc4):** Valida la integridad de la PROM mediante una función matemática de código de redundancia cíclica. Si un coeficiente estuviera corrupto, la altitud calculada fallaría estrepitosamente sin previo aviso.

3. Conversión de Datos Brutos (convert)

El sensor no entrega la presión en Pascales ni la temperatura en grados, sino valores crudos llamados D_1 (presión) y D_2 (temperatura), los cuales varían según la temperatura física del silicio del chip:

1. Envía el comando de conversión para D_1 o D_2 .
2. Espera unos **10 milisegundos** (`T_CONV_MS`) a que el conversor ADC interno termine el cálculo de alta resolución.
3. Lee el resultado de 24 bits (`b[0]`, `b[1]`, `b[2]`) y se asegura de que no devuelva cero (lo que indicaría un *timeout* por leer antes de tiempo).

4. Compensación Matemática y Orden Superior (ms5607_read)

Una vez obtenidos los datos crudos D_1 y D_2 , se aplican las fórmulas de compensación:

- **Uso de enteros de 64 bits (int64_t):** Al multiplicar variables como D_1 por C_1 o elevar potencias según las notas de aplicación del fabricante, los números intermedios **desbordan por completo los límites de 32 bits**. El uso de int64_t evita que el cálculo se corrompa.
- **Corrección de segundo orden:** A temperaturas bajas (por debajo de 20°C), el comportamiento del silicio cambia de forma no lineal. Durante el vuelo de un cohete, al atravesar capas atmosféricas muy frías (frecuentemente por debajo de -30°C o -40°C), **esta corrección es obligatoria**; de lo contrario, el error en el cálculo de la altitud barométrica sería de decenas de metros.

Finalmente, la función devuelve la temperatura en grados Celsius y la presión en Pascales listos para la navegación de la aviónica.

8. Ina226.c: El motor energético y el conteo de columbios

El problema de las baterías de LiPo: Las baterías de polímero de litio tienen una curva de descarga que es estable el 80% inicial y de repente caen en picado.

Cómo funciona el INA226:

- Se coloca una resistencia de valor ultrapequeño y alta precisión en serie (llamada *shunt*, por ejemplo de $10\text{ m}\Omega$).
- El chip mide la caída de voltaje infinitesimal que se produce al pasar la corriente a través de esa resistencia (ley de Ohm: $V = I \cdot R$).
- Mediante el registro de calibración, le dices al chip los parámetros de tu hardware para que él mismo traduzca esa caída de tensión directamente en miliamperios (mA).

Integración de corriente (Coulomb Counting): Al conocer la corriente exacta que consume la aviónica segundo a segundo, el software puede integrar esos datos en el tiempo ($Q = \int I \, dt$). Esto permite calcular con precisión matemática la energía real que le queda a la batería, evitando sorpresas fatales en vuelo.

1. Estructura y Registros Clave (#define)

- **Identificación (REG_MANUF_ID = 0xFE):** Espera el valor fijo 0x5449 (que corresponde a las letras "TI" de *Texas Instruments* en ASCII) para verificar que el chip está conectado correctamente.
- **Registros de Medida:**
 - **REG_BUS_V (0x02):** Mide la tensión de la barra (voltaje de la batería) con una escala fija de 1.25 mV por LSB.
 - **REG_CURRENT (0x04):** Mide la corriente que atraviesa la **resistencia shunt** (un componente ultrapequeño y de alta precisión colocado en serie). Al ser un valor con signo (int16_t), permite detectar la dirección del flujo.
 - **REG_CALIB (0x05):** Registro donde se escribe el valor matemático que le enseña al chip cómo traducir la caída de voltaje del shunt directamente a miliamperios.

2. Comunicación de 16 Bits (write16 y read16)

Como los registros del INA226 son de 16 bits y utilizan el formato **MSB primero** (*Most Significant Byte* primero), estas funciones auxiliares se encargan de empaquetar o desempaquetar los bytes para transmitirlos correctamente a través del bus I2C.

3. Inicialización y Calibración del Hardware (ina226_init)

1. **Comprobación de ID:** Lee el registro del fabricante para validar la presencia del sensor.
2. **Cálculo de Calibración:** Utiliza las fórmulas provistas por el fabricante para configurar el chip en función del hardware real de la placa:
 - **current_lsb_a:** Define la resolución mínima de la corriente ($I_{max} / 32768$).
 - **cal:** Aplica la fórmula de fábrica ($0.00512 / (current_lsb_a * r_shunt_ohm)$) para que el INA226 calcule la corriente de forma interna mediante la Ley de Ohm.
3. **Configuración de Muestreo (CONFIG_AVG16_CONT):** Configura el chip para promediar 16 muestras y realizar conversiones continuas tanto del shunt como del bus.

4. Lectura de Datos (ina226_read) y Coulomb Counting

1. Lee el voltaje del bus y lo multiplica por BUS_V_PER_LSB para obtener los voltios reales.
2. Lee el registro de corriente y lo convierte a un entero con signo (int16_t), multiplicándolo por la resolución calculada para obtener los amperios exactos.
3. **Utilidad en el cohete:** Al conocer la corriente consumida segundo a segundo, el software de la aviónica puede **integrar estos datos en el tiempo (Coulomb Counting)**. Esto permite calcular con precisión matemática cuánta energía real le queda a la batería, evitando sorpresas fatales en pleno vuelo ante la caída abrupta de las LiPo.

9. sensors.c:

Módulos fundamentales en el desarrollo de software embebido: la capa de unificación de sensores y su integración en el flujo principal.

Sensors.c :

-Función: **sensors_init** (no se detiene si un sensor falla (return a mitad) se registra en salud pero se continua)

-Lectura condicional: **sensors_read_all**; evalúa dinámicamente qué sensores están operativos. Si un sensor que funcionaba empieza a fallar durante la ejecución, se marca automáticamente como no disponible para evitar lecturas erróneas.

-Gestión de datos mediante estructuras clave:

sensors_health_t → Contiene el estado de salud de cada componente (ideal para enviar por telemetría y saber desde tierra que funciona)

sensors_snapshot_t → tomar fotografía de sensores junto a su marca de tiempo (t_ms), lista para ser guardada en la memoria Flash o consumida por el filtro Kalman.

10. Main.c

Dentro de USER CODE para no bloquear el resto de las cosas.

PRUEBA DE INTEGRACIÓN RÁPIDA:

En el arranque se ejecuta `sensors_init()`, seguido de un `printf` que muestra en consola el estado de cada sensor. Esto permite verificar el hardware en segundos.

Optimización numérica sin “float”:

Se imprimen enteros escalados (eg. Miligramos o pascals) para evitar habilitar el soporte de “float” en el `printf` de la librería estándar reducida. Esto ahorra una cantidad considerable de memoria programa (Flash) y RAM.

HAL_Delay(100): El uso de retardos bloqueantes (`HAL_Delay`) es solo una prueba de concepto inicial. En un entorno de vuelo real no es viable, ya que la IMU exige lecturas a alta frecuencia (1 kHz) guiadas por interrupciones físicas (pines como el PE6), y el barómetro no puede bloquear el procesador durante milisegundos esperando una conversión. La solución definitiva requiere pasar a FreeRTOS (tareas concurrentes) o a una máquina de estados no bloqueante.